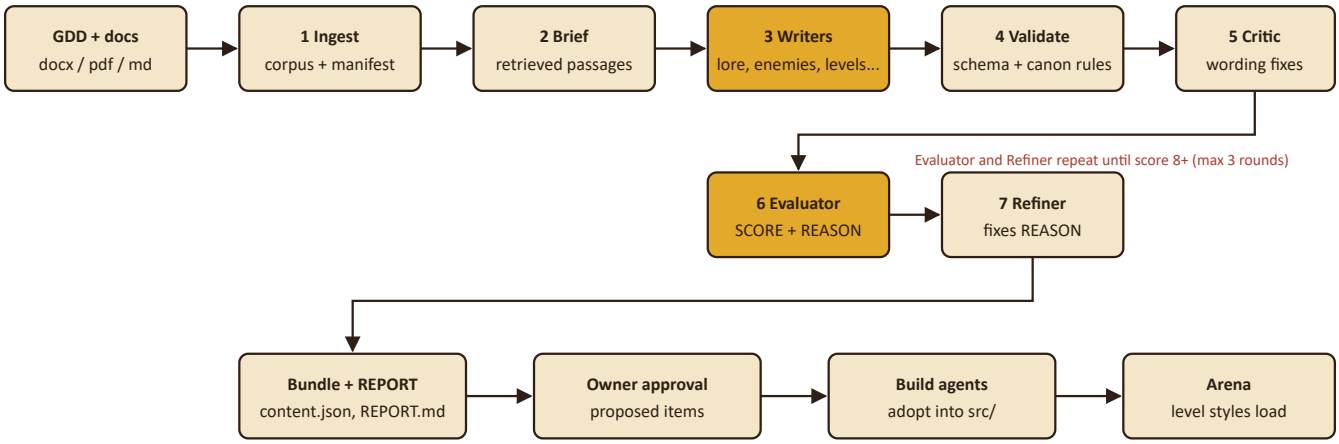
Pipeline Connection

The Rebellion: Fists of the Fallen City. How the GDD flows through agents, checks and the game.

1. The whole flow



2. Stages and files

Stage	Run by	Reads	Writes
1 Ingest	`pipeline/ingest.mjs`	every .docx/.pdf/.md/.txt in the repo	`pipeline/corpus/corpus.json`, `manifest.json`
2 Brief	`pipeline/brief.mjs`	corpus, schemas, house rules	`pipeline/briefs/<type>.md`
3 Writers	lore-writer, enemy-designer (first); level-designer, dialogue (after)	their brief	`content/<type>/*.json`
4 Validate	`pipeline/validate.mjs`	`content/`	errors back to the writer; `content/build/content.json` with `--bundle`
5 Critic	critic-agent	content, `design/lore/canon.md`	wording edits, `design/lore/critic-log.md`
6 Evaluator	evaluator-agent (read-only)	target, `design/art/style-guide.md`, canon	SCORE + REASON; `pipeline/evals/` via `pipeline/refine.mjs`
7 Refiner	refiner-agent	target + the REASON	fixed target, `design/lore/refiner-log.md`

3. The evaluate and refine loop

1. The evaluator scores the target 0 to 10 against the style guide and canon. Blockers cost 3, major 2, minor 1.

2. `pipeline/refine.mjs record` stores the score. `status` exits 0 (pass, score 8 or more), 1 (refine again) or 2 (cap of 3 rounds reached).
3. On 1, the refiner applies each REASON line, blockers first, changing nothing else, then the evaluator scores again.
4. On 2, the target goes to the owner. The director never fixes violations itself.

Last run: lore 8, enemies 10, levels 7 then 9, dialogue 9. Nothing capped.

4. Level style connection

Each level has its own style agent. Its outputs are what the arena reads, so this is where the pipeline meets the game.

Link	Detail
Owner direction	Given to `level-style-01-brick-red-alley-agent`; pasted verbatim into `design/levels/level-01-brick-red-alley.md`.
Rules it reads	`design/art/style-guide.md` (palette, lane, saturation), `design/lore/canon.md`, this level's entry in `content/levels/`.
Machine-readable style	`content/levels/styles/level-01-brick-red-alley.json`: background path, colour overlay, ambient light settings.
Art	`assets/levels/level-01-brick-red-alley/background.svg` (3840x720, tileable).
Game loading	`src/levels/levelData.js` sets `STYLE_DIR = content/levels/styles/`; `src/scenes/ArenaScene.js` loads the per-level background and falls back to the shared placeholder if no style file exists. `src/levels/ambientData.js` reads the optional `ambient` block.
Off limits	Waves, timer and intro/outro text stay in `content/levels/levels.json` and `rust-gdd-levels.json`; the style agent reports needed changes instead of editing them.

5. How to run it

Goal	Command
Ingest, briefs, validate, report (no model calls)	`npm run content`
Same, plus the four writers headlessly	`npm run content:generate`
Validate and bundle	`node pipeline/validate.mjs --bundle`
Score loop status for one target	`node pipeline/refine.mjs status "content/levels"`
Whole pipeline with all agents	Ask Claude Code: "use content-director-agent to generate content from the GDD"

6. Rules that hold across the pipeline

- Primary source is `GameDesignDocument/`; `design/` is derived; everything else is reference.
- Every item is `canon` (in the GDD) or `proposed` (with a `gddGap` note). Proposals need owner approval.
- The validator bans "Rush", "Fallen" alone, Rust as a place, gore, killing words and gloating copy.
- Generated content is data only. Adopting it into `src/` is a separate, owner-approved step.